Task 1: Eliminating Repetitive Logic

Original code had repeated area and perimeter calculations. Used AI suggestion to create functions for reuse.

Refactored Code (task1_refactored.py):

```
def calculate_area(length, width):
    """Calculate area of rectangle."""
    return length * width

def calculate_perimeter(length, width):
    """Calculate perimeter of rectangle."""
    return 2 * (length + width)

# First rectangle
length = 8
width = 4
print("Area:", calculate_area(length, width))
print("Perimeter:", calculate_perimeter(length, width))

# Second rectangle
length = 12
width = 6
print("Area:", calculate_area(length, width))
print("Perimeter:", calculate_perimeter(length, width))
```

Output:

```
$ python task1_refactored.py
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Task 2: Modularizing Inline Calculations

Repeated discount logic extracted into function per AI refactor suggestion.

Refactored Code (task2_refactored.py):

```
def apply_discount(amount, rate=0.10):
    """Apply discount to amount and return final amount."""
    discount = amount * rate
    return amount - discount

# First amount
amount = 1000
print("Final Amount:", apply_discount(amount))

# Second amount
amount = 2000
print("Final Amount:", apply_discount(amount))
```

Output:

```
$ python task2_refactored.py
Final Amount: 900.0
Final Amount: 1800.0
```

Task 3: Improving Loop and Conditional Performance

Nested loops inefficient. AI suggested set for O(1) lookups.

Original Execution Test:

```
$ time python task3_original.py
Charan found
Esha not found
Bhavya found
real    0m0.000s
```

Refactored Code (task3_refactored.py):

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

# Convert to set for fast lookup
students_set = set(students)
```

```
for name in check_list:
    if name in students_set:
        print(name, "found")
    else:
        print(name, "not found")
```

Terminal Output and Test:

```
$ python task3_refactored.py
Charan found
Esha not found
Bhavya found
$ time python task3_refactored.py
real    0m0.000s
```

Faster lookups, same output. Set improves performance for larger lists.

Task 4: Replacing Hardcoded Values with Constants

Hardcoded numbers replaced with constants as AI recommended.

Refactored Code (task4_refactored.py):

```
PRINCIPAL1 = 1000
PRINCIPAL2 = 2000
TIME = 2
RATE = 5  # percent

def simple_interest(p, t, r):
    return (p * t * r) / 100

print("Simple Interest:", simple_interest(PRINCIPAL1, TIME, RATE))
print("Simple Interest:", simple_interest(PRINCIPAL2, TIME, RATE))
```

Terminal Output:

```
$ python task4_refactored.py
Simple Interest: 100.0
Simple Interest: 200.0
```

Matches original. Easy to update RATE or TIME now.

Task 5: Enhancing Variable Naming and Code Clarity

Renamed variables and added comments based on AI suggestions for readability.

Refactored Code (task5_refactored.py):

```
first_number = 50   # Starting value for addition
second_number = 30  # Value to add to first number
total_sum = first_number + second_number  # Sum of the two numbers

print(total_sum)  # Output the result
```

Output:

```
$ python task5_refactored.py
80
```

Same output. Much clearer what each variable means.